

Лабораторная работа № 6.

Аутентификация во frontend-приложении

Тема: Auth state, protected pages, route guards и role-aware UI

Цель: Реализовать во frontend Internet Shop единое состояние аутентификации, hash-routing, protected pages, route guards, role-aware navigation, обработки 401/403 и logout.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте frontend из лабораторной работы №5. Backend должен поддерживать POST /api/auth/login, GET /api/auth/me, GET /api/orders/my, JWT, роли user/admin и middleware requireAuth.

Route guard не является защитой API

Frontend guard управляет навигацией и интерфейсом. Реальный доступ к данным по-прежнему проверяется backend через JWT, role и ownership.

1. Расширьте структуру frontend

Структура проекта

```
frontend/  
  index.html  
  app.js  
  api.js  
  authState.js  
  router.js  
  views.js  
  config.js  
  styles.css
```

2. Обновите authState.js

frontend/authState.js

```
const state = {  
  status: "guest",  
  user: null,  
  accessToken: null  
};  
  
export function getAuthState() {  
  return state;  
}  
  
export function setLoading() {  
  state.status = "loading";  
}  
  
export function setAuthenticated(user, accessToken) {  
  state.status = "authenticated";  
}
```

```

    state.user = user;
    state.accessToken = accessToken;
  }

  export function setGuest() {
    state.status = "guest";
    state.user = null;
    state.accessToken = null;
  }

  export function getAccessToken() {
    return state.accessToken;
  }

```

Frontend теперь явно различает состояния `loading`, `guest` и `authenticated`.

3. Создайте контейнер приложения

```

frontend/index.html

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Internet Shop</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <nav id="navigation"></nav>
  </header>

  <main id="app"></main>

  <script type="module" src="app.js"></script>
</body>
</html>

```

4. Создайте views.js

```

frontend/views.js

export function renderHome(app) {
  app.innerHTML = `
    <h1>Internet Shop</h1>
    <p>Главная страница.</p>
  `;
}

export function renderLogin(app) {
  app.innerHTML = `
    <h1>Вход</h1>
    <form id="login-form">
      <input id="email" type="email" required placeholder="Email">
      <input id="password" type="password" required placeholder="Password">
      <button type="submit">Войти</button>
    </form>
    <p id="login-error"></p>
  `;
}

```

```

    `;
  }

  export function renderProfile(app, user) {
    app.innerHTML = `
      <h1>Профиль</h1>
      <p>${user.name}</p>
      <p>${user.email}</p>
      <p>Роль: ${user.role}</p>
    `;
  }

  export function renderForbidden(app) {
    app.innerHTML = `
      <h1>403</h1>
      <p>Недостаточно прав</p>
    `;
  }

  export function renderLoading(app) {
    app.innerHTML = "<p>Загрузка...</p>";
  }
}

```

5. Создайте route guards

```

frontend/router.js

import { getAuthState } from "../authState.js";

export function requireUser() {
  const auth = getAuthState();

  if (auth.status !== "authenticated") {
    location.hash = "#/login";
    return false;
  }

  return true;
}

export function requireAdmin() {
  const auth = getAuthState();

  if (auth.status !== "authenticated") {
    location.hash = "#/login";
    return false;
  }

  if (auth.user.role !== "admin") {
    return false;
  }

  return true;
}

```

Проверка роли дублируется backend

`requireAdmin()` нужен для UX. Admin API endpoints всё равно должны использовать

requireAuth и requireRole("admin") на сервере.

6. Реализуйте router

frontend/router.js

```
import {
  renderHome,
  renderLogin,
  renderProfile,
  renderForbidden
} from "./views.js";
import { getAuthState } from "./authState.js";

export function renderRoute() {
  const app = document.querySelector("#app");
  const route = location.hash || "#/";
  const auth = getAuthState();

  if (route === "#/login") {
    if (auth.status === "authenticated") {
      location.hash = "#/profile";
      return;
    }

    renderLogin(app);
    return;
  }

  if (route === "#/profile") {
    if (!requireUser()) return;
    renderProfile(app, auth.user);
    return;
  }

  if (route === "#/orders") {
    if (!requireUser()) return;
    app.innerHTML = "<h1>Мои заказы</h1><div id='orders'></div>";
    return;
  }

  if (route === "#/admin") {
    if (!requireAdmin()) {
      renderForbidden(app);
      return;
    }

    app.innerHTML = "<h1>Admin Panel</h1>";
    return;
  }

  renderHome(app);
}
```

7. Подключите hashchange

frontend/app.js

```
import { renderRoute } from "./router.js";
```

```
window.addEventListener("hashchange", renderRoute);
window.addEventListener("DOMContentLoaded", renderRoute);
```

Переходы выполняются через адреса вида `#/login`, `#/profile`, `#/orders` и `#/admin`.

8. Реализуйте role-aware navigation

frontend/app.js

```
import { getAuthState } from "../authState.js";

export function renderNavigation() {
  const nav = document.querySelector("#navigation");
  const auth = getAuthState();

  if (auth.status === "authenticated") {
    nav.innerHTML = `
      <a href="#/">Главная</a>
      <a href="#/orders">Мои заказы</a>
      <a href="#/profile">Профиль</a>
      ${auth.user.role === "admin" ? '<a href="#/admin">Admin</a>' : ''}
      <button id="logout-button">Выйти</button>
    `;
  } else {
    nav.innerHTML = `
      <a href="#/">Главная</a>
      <a href="#/login">Войти</a>
    `;
  }
}
```

9. Подключите login form

frontend/app.js

```
import { apiFetch } from "../api.js";
import { setAuthenticated } from "../authState.js";

async function login(email, password) {
  const data = await apiFetch("/auth/login", {
    method: "POST",
    body: {
      email,
      password
    }
  });
}

setAuthenticated(data.user, data.accessToken);
renderNavigation();
location.hash = "#/orders";
}

document.addEventListener("submit", async (event) => {
  if (event.target.id !== "login-form") return;

  event.preventDefault();
```

```

const email = document.querySelector("#email").value;
const password = document.querySelector("#password").value;
const errorBox = document.querySelector("#login-error");

try {
  await login(email, password);
} catch (error) {
  errorBox.textContent = error.message;
}
});

```

10. Проверьте protected profile

1. Откройте приложение как guest.
2. Вручную введите `#/profile`.
3. Убедитесь, что frontend перенаправляет на `#/login`.
4. Выполните успешный login.
5. Снова откройте `#/profile`.
6. Убедитесь, что отображаются данные User.

11. Проверьте Admin page

7. Войдите как обычный user.
8. Попробуйте открыть `#/admin` вручную.
9. Убедитесь, что отображается страница Недостаточно прав.
10. Выйдите из системы.
11. Войдите как admin.
12. Убедитесь, что в navigation появился пункт Admin.
13. Откройте `#/admin`.

12. Добавьте загрузку заказов

```

frontend/app.js

async function loadOrdersPage() {
  const container = document.querySelector("#orders");

  if (!container) return;

  container.textContent = "Загрузка...";

  try {
    const orders = await apiFetch("/orders/my");

    container.innerHTML = "";

    for (const order of orders) {
      const item = document.createElement("p");
      item.textContent = `Заказ #${order.id}: ${order.status}`;
      container.append(item);
    }
  } catch (error) {
    container.textContent = error.message;
  }
}

```

13. Вызывайте загрузку после render route

```
frontend/app.js

window.addEventListener("hashchange", () => {
  renderRoute();

  if (location.hash === "#/orders") {
    loadOrdersPage();
  }
});
```

14. Реализуйте централизованный 401

```
frontend/api.js

if (response.status === 401) {
  setGuest();
  renderNavigation();
  location.hash = "#/login";

  throw new APIError(
    "Сессия истекла. Войдите снова.",
    401
  );
}
```

При 401 пользователь больше не считается authenticated.

15. Не выполняйте logout при 403

Пример обработки admin action

```
try {
  await apiFetch("/products/15", {
    method: "DELETE"
  });
} catch (error) {
  if (error.status === 403) {
    alert("Недостаточно прав");
  }
}
```

403 не разрушает Authentication

Пользователь остаётся authenticated и может продолжать работать с разрешёнными страницами.

16. Реализуйте logout

```
frontend/app.js

import { setGuest } from "../authState.js";

document.addEventListener("click", (event) => {
```

```
if (event.target.id !== "logout-button") return;

setGuest();
renderNavigation();
location.hash = "#/";
});
```

17. Проверьте поведение после reload

14. Выполните login.
15. Откройте `#/orders`.
16. Нажмите Refresh.
17. Убедитесь, что in-memory auth state исчез.
18. Попробуйте снова открыть protected page.
19. Frontend должен перенаправить на login.

Не исправляйте это localStorage

Для этой лабораторной потеря состояния после reload является ожидаемой. Не переносите JWT в localStorage или sessionStorage.

18. Изучите концепцию auth bootstrap

В production-приложении с server-managed HttpOnly cookie frontend при старте может вызвать GET `/api/auth/me`. Успех переводит state в authenticated, а 401 — в guest. JavaScript при этом не обязан читать credential.

Пока только концепция

Cookie-based bootstrap, credentials: include, CORS и CSRF будут подробно разобраны в следующей лабораторной работе.

19. Самостоятельное задание

20. Добавьте route `#/profile`.
21. Добавьте route `#/orders`.
22. Добавьте route `#/admin`.
23. Реализуйте `requireUser()`.
24. Реализуйте `requireAdmin()`.
25. Скрывайте Admin link для обычного user.
26. Проверьте ручной переход user на `#/admin`.
27. Обработывайте 401 переходом на login.
28. При 403 оставляйте пользователя authenticated.
29. Проверьте logout.
30. Проверьте reload страницы.
31. Не используйте localStorage/sessionStorage для JWT.

Требования к результату

- Frontend имеет единый auth state.
- Используются состояния guest/loading/authenticated.
- Реализован hash router.
- Protected pages используют route guards.

- User открывает profile и orders.
- Admin UI доступен только роли admin.
- Navigation зависит от auth state.
- 401 очищает Authentication.
- 403 не выполняет logout.
- Logout очищает state и UI.
- Reload уничтожает in-memory JWT.
- JWT не сохраняется в Web Storage.

Критерии оценивания

№	Критерий
1	Создан authState со status/user/accessToken
2	Реализован hash router
3	Реализован requireUser()
4	Реализован requireAdmin()
5	Guest перенаправляется с protected page на login
6	Authenticated user открывает profile
7	Authenticated user открывает orders
8	Обычный user не получает admin UI
9	Admin получает admin UI
10	Navigation зависит от auth state
11	401 переводит frontend в guest
12	403 не очищает auth state
13	Logout очищает protected UI
14	JWT не хранится в localStorage/sessionStorage
15	Студент объясняет, почему frontend guard не заменяет backend authorization